ARCHITECTURE-STYLES-DESIGN-PATTERNS

아키텍처 스타일·GoF 패턴·프록시

연관 기출 · 131-2-6, 129-2-6, 136-1-5

한눈에 이해

아키텍처 스타일은 시스템을 어떻게 나누고 연결할지 정합니다. 디자인 패턴은 반복되는 설계 문제에서 각 객체가 맡을 역할을 설명합니다.

정의와 핵심 원리

아키텍처 스타일은 구성요소·연결자·제약으로 시스템 전체의 조직을 규정하고, 디자인 패턴은 특정 맥락에서 반복되는 설계 문제의 역할·협력·결과를 재사용 가능한 형태로 설명합니다.

• 계층형은 표현·업무·데이터 책임을 나누어 변경과 의존을 관리하지만 계층 관통 호출과 불필요한 지연을 통제해야 합니다. 파이프-필터는 단계별 변환을 연결해 재사용·병렬화를 돕지만 공유 상태와 대화형 처리가 어렵습니다. 클라이언트-서버는 요청자와 서비스 제공자를 분리하되 서버 병목과 네트워크 장애를 고려합니다.

• GoF 생성 패턴은 객체 생성 변경을 감추는 Factory Method·Abstract Factory·Builder 등을, 구조 패턴은 구성·접근을 바꾸는 Adapter·Decorator·Proxy 등을, 행위 패턴은 알고리즘과 협력을 바꾸는 Strategy·Observer·Command 등을 다룹니다.

• Proxy는 Subject 인터페이스를 구현하고 RealSubject를 대신 참조해 접근을 통제합니다. 원격 프록시, 지연 생성의 가상 프록시, 권한 검사의 보호 프록시가 대표적이며 호출자는 같은 계약을 사용합니다.

• 리팩토링은 외부 동작을 유지하는 변화의 과정이고 디자인 패턴은 리팩토링으로 적용할 수 있는 설계 구조입니다. 모두 유지보수성을 도울 수 있으나 패턴 도입은 클래스·간접호출을 늘릴 수 있으므로 실제 변화 압력이 있을 때 선택합니다.

구조·비교·판단

구분	원리·특징	적용·판단
스타일	시스템 수준 구성과 제약	계층형·파이프필터·클라이언트 서버
디자인 패턴	클래스·객체의 역할과 협력	생성·구조·행위 문제 해결
리팩토링	행동을 보존하는 작은 변환	필요할 때 패턴 구조로 이동

적용 예제

강사 창작 적용 예시: 대용량 문서 프록시

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 목록 화면에서 실제 PDF 전체를 즉시 읽어 지연됩니다.

1. Document.display()를 Subject 계약으로 정합니다.
2. DocumentProxy는 파일 위치와 제목만 보관하고 최초 display 시 RealDocument를 생성합니다.
3. 로그인 권한을 먼저 확인하고 생성 실패를 같은 계약의 오류로 반환합니다.

결과·해석: 실제로 열람한 문서만 불러오고 접근 권한은 프록시에서 확인할 수 있습니다.

전제·주의: 지연 로딩은 총비용을 제거하지 않으며 첫 열람 지연과 동시 생성 제어가 필요합니다.

답안 골격과 혼동 방지

스타일과 패턴 비교 → 스타일 3종과 GoF 분류·대표 예 → Proxy 객체 연결 및 리팩토링과의 관계

- 모든 프록시가 네트워크 리버스 프록시를 의미하지 않습니다.
- 패턴 명칭 나열 대신 해결하는 변경 문제를 적습니다.

내용을 가리고 회상

1. Proxy와 Decorator의 주된 의도는 어떻게 다릅니까?
2. 파이프-필터에 공유 상태가 많으면 무엇이 어려워집니까?
3. 리팩토링과 디자인 패턴의 공통점과 차이점을 말하십시오.

적용 기준과 근거

SWEBOK의 스타일·패턴 구분은 확인했습니다. GoF 원저 전체를 열람하지 않아 세부 분류·표기의 규범적 대조는 추가 확인 필요합니다.

내용 검토 상태: 부분 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- Guide to the SWEBOK v4.0a - IEEE Computer Society / Hironori Washizaki 편; 2026-08; 확인 2026-11-05.
- Refactoring - Martin Fowler; 지속 갱신 문서; 확인 2026-11-05.